Universidad de La Sabana

UNIVERSIDAD DE LA SABANA
MAESTRÍA EN INTELIGENCIA ARTIFICIAL

REPORTE TÉCNICO: PROTOTIPO DE DETECCIÓN DE MOVILIDAD URBANA DESDE DRON MEDIANTE TRANSFERENCIA DE APRENDIZAJE Y FINE-TUNING (YOLO11n)

Autores: Alejandro Moncada, Edgar Julián Méndez Ortegón, Noel Eduardo Pérez Barrios (Grupo 9)
Asignatura: Técnicas Avanzadas de Modelado en IA · Actividad 3, Unidad 3 · Octubre de 2026
Código: github.com/AlejoMoncada/yolo-realtime-pipeline/tree/AlejoMoncada/g9-act3-transfer-learning

1. Resumen ejecutivo y preparación de los datos

1.1 Resumen ejecutivo. Tomamos YOLO11n, preentrenado en COCO, y lo adaptamos para detectar personas y vehículos en imágenes de dron (VisDrone2019-DET). Esperábamos que el modelo COCO sirviera como punto de partida razonable, porque ya conoce personas, carros y buses. No fue así: sin ajuste obtuvo apenas **6,2 %** de mAP@50 en test. Desde arriba y con pocos píxeles de tamaño, los objetos casi no se parecen a los de COCO. Con transferencia de aprendizaje en dos etapas (10 épocas con el backbone congelado y 30 de ajuste completo) llegamos a **24,9 %** de mAP@50 y **13,0 %** de mAP@50-95, unas 4,0 veces más. Entrenado desde cero con el mismo presupuesto de 40 épocas, el mismo modelo solo llega a 15,5 %, así que el preentrenamiento aporta 9,4 puntos porcentuales (pp). Todas las cifras salen de los artefactos de entrenamiento del repositorio, y no usamos el conjunto de prueba ni para entrenar ni para elegir modelos.

1.2 Carga del dataset, particiones y control de fuga

Leemos las imágenes directamente de los zips oficiales de VisDrone2019-DET y convertimos sus anotaciones al formato YOLO normalizado (`clase xc yc w h` en [0, 1]). Agrupamos las 10 categorías originales en 6 clases que también existen en COCO: *persona* (pedestrian y people), *bicicleta*, *carro* (car y van), *moto*, *bus* y *camión*. Elegimos esa agrupación para poder evaluar el modelo COCO sin reentrenarlo y así tener una línea base justa. Los triciclos y las regiones que VisDrone marca como ignoradas no los borramos: los pintamos de gris. Si solo quitáramos sus etiquetas, el modelo aprendería a tratarlos como fondo, y cada detección sobre ellos contaría como falso positivo.

Para las particiones usamos los splits oficiales, que vienen de **secuencias de video distintas**. En la práctica es un *group split*: dos cuadros casi iguales del mismo video nunca quedan uno en train y otro en test. De train usamos 2.000 de las 6.471 imágenes. Entrenamos en un computador con chip Apple M4 y menos de 5 GB libres en disco, donde cada época sobre esas 2.000 imágenes tardaba entre dos y cuatro minutos; con el conjunto completo, las corridas no habrían cabido en el tiempo de la actividad. Para no perder variedad de escenas, tomamos las imágenes de forma alternada entre secuencias, con semilla fija. Val y test los usamos completos.

Split	Imágenes	persona	bicicleta	carro	moto	bus	camión	Total	Uso
train	2.000	43.732	4.748	58.520	12.371	1.409	3.819	124.599	ajuste de pesos
val	548	13.969	1.287	16.039	4.886	251	750	37.182	selección de época y curvas
test	1.610	27.382	1.302	33.845	5.845	2.940	2.659	73.973	solo el reporte final

1.3 Normalización y aumentación aleatoria

Reducimos las imágenes a 1.024 px en el lado mayor. En el entrenamiento, Ultralytics las lleva a 640×640 con relleno gris (letterbox), pasa de BGR a RGB y escala los píxeles a [0, 1]. No aplicamos la media y la desviación de ImageNet, porque YOLO11 se preentrenó con la escala [0, 1] y cambiarla rompería la transferencia. La aumentación aleatoria va solo en train: mosaic, volteo horizontal (p = 0,5), variación de color HSV, escala (±50 %) y traslación (±10 %). Ultralytics apaga el mosaic en las últimas 10 épocas de cada corrida, así que la Etapa 1, de 10 épocas, se entrenó sin él. El desbalance es fuerte: *carro* y *persona* suman el 82 % de las instancias de train, y *bus* apenas el 1,1 %.

2. Arquitectura y estrategia de transferencia de aprendizaje

2.1 Modelo preentrenado y capa de salida propia

YOLO11n tiene 2,61 millones de parámetros. Lo cargamos explícitamente desde los pesos de COCO con `YOLO("yolo11n.pt")`. Su backbone ocupa los módulos 0 a 10, el cuello PAN-FPN va del 11 al 22 y la cabeza de detección es el módulo 23. Nuestra capa de salida propia es la parte de clasificación de esa cabeza: reemplazamos las 80 salidas de COCO por 6 nuevas, inicializadas al azar, una por clase. La parte que predice la posición de las cajas la conservamos, porque la geometría de una caja no depende de la clase.

2.2 Función de pérdida

$$L_{total} = 7{,}5 \cdot L_{CIoU} + 0{,}5 \cdot L_{BCE} + 1{,}5 \cdot L_{DFL}, \quad L_{BCE} = -\Sigma_c \left[y_c \log p_c + (1 - y_c) \log(1 - p_c) \right]$$

La clasificación se entrena con entropía cruzada binaria por clase, y CIoU y DFL se encargan de la localización. Optimizamos con AdamW, la versión de Adam con *weight decay* desacoplado.

2.3 Esquema de transferencia en dos etapas

En la **Etapa 1** congelamos todo el backbone (`freeze=11`) y entrenamos solo el cuello y la cabeza durante 10 épocas con LR 1e-3. La cabeza nueva arranca al azar y al principio produce gradientes grandes; con el backbone libre, esos gradientes dañarían los filtros que trae de COCO. Verificamos el congelamiento comparando los pesos módulo por módulo: el backbone sale de la Etapa 1 exactamente igual a como entró, mientras que el cuello y la cabeza sí cambian.

En la **Etapa 2** descongelamos todo (`freeze=0`) y entrenamos 30 épocas más con un LR cinco veces menor, 2e-4, que baja hasta 1e-5 con decaimiento coseno. Descongelamos desde la capa 0 porque la vista aérea cambia incluso los rasgos más básicos que aprenden las primeras capas: objetos diminutos, vista desde arriba y texturas de techos y pavimento.

2.4 Configuración de hiperparámetros

Hiperparámetro	Etapa 1	Etapa 2	Justificación
Optimizador	AdamW (β_1 = 0,937; β_2 = 0,999), weight decay 5e-4		Adam con decaimiento desacoplado.
LR inicial → final	1e-3 → 1e-5, lineal	2e-4 → 1e-5, coseno	Paso grande para la cabeza nueva; paso fino para no dañar el backbone.
Warmup	1 época	ninguno	La Etapa 2 parte de pesos ya estables.
Capas entrenables	11 a 23	0 a 23	Congelar el backbone (0 a 10) estabiliza la cabeza nueva.
Comunes a todas las corridas: imagen de 640, batch 16, semilla 0, `deterministic=True` y 2.000 imágenes de train. Guardamos los pesos con mejor mAP@50-95 en val y no usamos early stopping, para que todas las corridas tengan el mismo presupuesto.			

> **Carga explícita de pesos** (`experiments.py`): `YOLO("yolo11n.pt").train(freeze=11, epochs=10, lr0=1e-3, optimizer="AdamW", seed=0, ...)` para la Etapa 1 y `YOLO("runs/act3/stage1/weights/best.pt").train(freeze=0, epochs=30, lr0=2e-4, cos_lr=True, ...)` para la Etapa 2. Ultralytics copia del modelo COCO todos los pesos que tienen la misma forma, es decir, todo menos la salida de clases.

3. Entrenamiento y curvas de aprendizaje

3.1 Evolución de pérdidas y precisión (train contra validación)

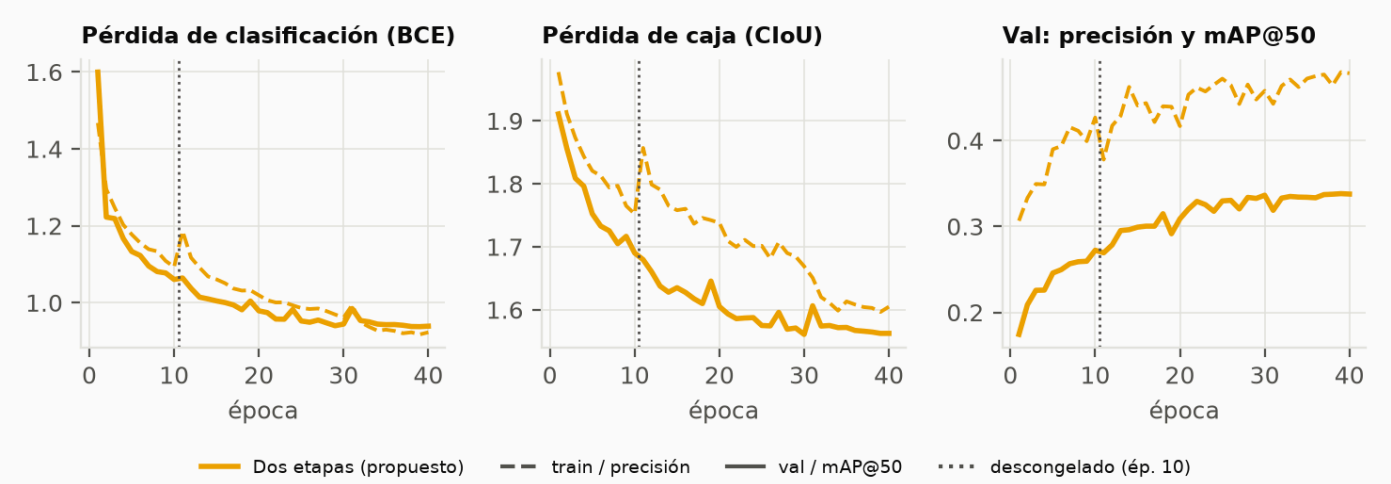


Figura 1. Curvas por época del modelo propuesto. En las pérdidas, la línea discontinua es train y la continua es validación; en el panel derecho, la discontinua es precisión y la continua es mAP@50, ambas en validación. La línea punteada marca el descongelamiento.

Durante la Etapa 1, con el backbone fijo, el mAP@50 en val sube de 17,4 % a 27,2 %. Al descongelar sigue subiendo hasta 33,8 %, lo que muestra que ajustar el backbone sí aporta. Las pérdidas de validación bajan junto con las de train y no se separan, así que no vemos sobreajuste fuerte. En la época 11 la pérdida de train da un salto. No es inestabilidad: la Etapa 1 corrió sin mosaic y la Etapa 2 lo vuelve a activar, lo que hace más difíciles las imágenes de train. La pérdida de validación, que no usa aumentación, no salta. Cuando el mosaic se apaga de nuevo (época 31) hay una caída breve del mAP que se recupera en una o dos épocas.

Ultralytics no calcula precisión ni mAP sobre train en cada época, solo las pérdidas. Por eso la curva de precisión de la Figura 1 es de validación. Para comparar train con validación medimos el mAP en los tres conjuntos con los mejores pesos (segunda tabla de 3.2).

3.2 Hitos del entrenamiento y control de sobreajuste

Hito	Época	box train	box val	cls train	cls val	Precisión val (%)	mAP@50 val (%)
Inicio de la Etapa 1 (backbone congelado)	1	1,98	1,91	1,47	1,60	30,7	17,4
Fin de la Etapa 1	10	1,75	1,69	1,09	1,06	42,7	27,2
Descongelamiento (inicio de la Etapa 2)	11	1,86	1,68	1,19	1,06	37,8	26,9
Se apaga el mosaic	31	1,65	1,61	0,99	0,98	44,3	31,9
Mejores pesos (máx. mAP@50-95 en val)	**37**	**1,60**	**1,57**	**0,92**	**0,94**	**47,7**	**33,7**
Cierre	40	1,60	1,56	0,92	0,94	47,8	33,8

Modelo (mejores pesos)	mAP@50 train	mAP@50 val	mAP@50 test
Etapa 1 (backbone congelado)	28,5	26,5	19,2
Dos etapas 10+30 (propuesto)	39,6	32,9	24,9

Con 2.000 imágenes es normal que train quede por encima. La caída de val a test indica cuánto generaliza el modelo a secuencias que nunca vio.

4. Evaluación en el conjunto de prueba

4.1 Comparación contra los modelos base

Modelo	mAP@50 (%)	mAP@50-95 (%)	Precisión (%)	Recall (%)	Épocas	Entrenamiento M4 (min)
COCO sin ajuste	6,2	3,1	23,3	8,8	0	—
Desde cero	15,5	7,3	29,5	18,3	40	—
Etapa 1 (backbone congelado)	19,2	10,0	33,5	21,8	10	25
Dos etapas 10+30 (propuesto)	**24,9**	**13,0**	**39,4**	**26,8**	**40**	**102**
Mejora frente a COCO sin ajuste	*×4,0 (+18,7 pp)*	*×4,2*				
Mejora frente a desde cero	*×1,6 (+9,4 pp)*	*×1,8*				

Promedios sobre las 6 clases. Precisión y recall se toman en el punto de máximo F1. Todas las corridas comparten datos, semilla, tamaño de imagen, batch, optimizador y número de épocas. No reportamos *accuracy* porque en detección no hay verdaderos negativos que contar; su equivalente son la precisión, el recall y el F1 por clase.

Al comparar clase por clase con el modelo entrenado desde cero, el preentrenamiento mejora las seis. Donde más pesa es en las clases con pocos ejemplos: *camión* multiplica su AP@50 por 2,7 y *bicicleta* por 2,5. En *carro*, la clase más abundante, la ganancia es de solo 1,2 veces. Cuando faltan datos, lo aprendido en COCO reemplaza ejemplos que el modelo no tiene.

4.2 Métricas por clase del modelo propuesto

Clase	Precisión	Recall	F1	AP@50	AP@50-95	AP@50 COCO sin ajuste	Instancias
persona	0,466	0,171	0,250	0,170	0,061	0,073	27.382
bicicleta	0,270	0,040	0,070	0,047	0,017	0,008	1.302
carro	0,541	0,623	0,579	0,609	0,341	0,200	33.845
moto	0,324	0,201	0,248	0,159	0,054	0,022	5.845
bus	0,511	0,329	0,400	0,323	0,205	0,027	2.940
camión	0,252	0,246	0,249	0,187	0,104	0,041	2.659
macro	**0,394**	**0,268**	**0,299**	**0,249**	**0,130**	**0,062**	**73.973**

4.3 Matriz de confusión y análisis de errores

La matriz del modelo propuesto (Figura 2) muestra que el error dominante no es confundir clases sino no detectar el objeto. Perdemos el 83 % de las personas, el 82 % de las bicicletas y el 78 % de las motos, frente al 38 % de los carros. Ese orden coincide con el tamaño de los objetos. A la entrada de 640 px, la mitad de las personas mide menos de 5,5 px de lado, mientras que un bus típico mide 24,5 px. Objetos tan pequeños ocupan menos de una celda en el mapa más fino que tiene YOLO11n.

Las confusiones entre clases se concentran en los vehículos grandes: 445 camiones salen como carro, el 17 % del total, y 385 carros salen como camión. Visto desde arriba, un furgón, que agrupamos como carro, y un camión pequeño casi solo se diferencian por el largo; esa frontera la pusimos nosotros al agrupar las clases. Algo parecido pasa con los vehículos de dos rue-

das. El 11 % de las bicicletas se predice como moto, y hay cruces entre persona y moto en ambos sentidos. VisDrone anota por separado al conductor y a su vehículo, con cajas casi superpuestas, y desde el dron el conductor tapa casi toda la moto.

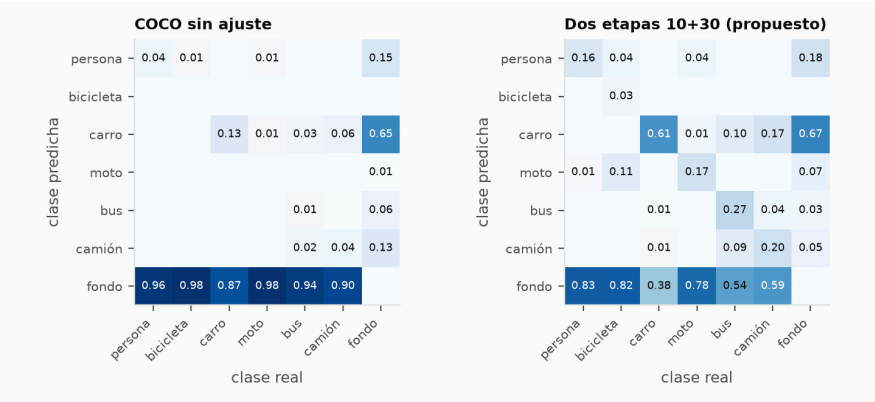


Figura 2. Matrices de confusión en test (confianza 0,25), normalizadas por columna: cada columna es una clase real. La fila "fondo" son objetos no detectados; la columna "fondo", detecciones sin objeto real.

Los falsos positivos suman 17.649, sobre todo de carro y persona. Al revisar las imágenes con más falsos positivos (Figura 3) encontramos que la mayoría son objetos reales que nadie anotó: carros en vías del fondo y personas en multitudes de noche. El resto cae sobre cosas con silueta parecida, como papeleras, troncos o postes. Por eso creemos que la precisión real es más alta que la medida, y que el problema de fondo es el recall de objetos pequeños. La mejor clase es *carro* (AP@50 de 0,609), la más abundante y la de objetos más grandes; la peor es *bicicleta* (0,047), pequeña y escasa.



Figura 3. Imágenes de test con más falsos positivos. Verde: anotaciones. Rojo: detecciones sin anotación que coincida.

5. Conclusiones, limitaciones y reproducibilidad

La transferencia funciona, pero en este dominio no basta con entrenar la cabeza. Con el backbone congelado pasamos de 6,2 % a 19,2 % de mAP@50 en test, y solo al ajustar también el backbone llegamos a 24,9 %. La vista aérea cambia los rasgos desde las primeras capas.

El límite actual es la escala, no la semántica: la matriz tiene muchos más objetos perdidos que confundidos. Lo siguiente que probaríamos es entrenar con imágenes más grandes (960 a 1.280 px) o recortadas en mosaicos, agregar una salida de mayor resolución para objetos pequeños y usar las 6.471 imágenes de train.

Este trabajo tiene límites claros. Usamos un subconjunto de train y una sola semilla, sin intervalos de confianza. En la GPU del M4 algunas operaciones no son deterministas aunque fijemos la semilla. Además, los triciclos quedaron fuera del problema.

Reproducibilidad: Python 3.12.11, PyTorch 2.14.1, Ultralytics 8.4.176 y semilla 0, en Apple M4 (MPS), 16 GB RAM, macOS; la línea base desde cero se entrenó con el mismo código, datos, semilla y configuración en una NVIDIA GTX 1660 Super (PyTorch 2.14.1+cu126), así que su mAP es comparable pero su tiempo no. Todo se regenera con `python -m transfer_learning.prepare_data --download`, `python -m transfer_learning.experiments all`, el notebook `transfer_learning/act3_transfer_learning.ipynb` y `python -m transfer_learning.report.build_report`.

6. Referencias

1. Zhu, P. et al. (2021). Detection and Tracking Meet Drones Challenge. *IEEE TPAMI*, 44(11), 7380-7399 (VisDrone).
2. Jocher, G., Qiu, J. (2024). *Ultralytics YOLO11*. https://docs.ultralytics.com
3. Yosinski, J. et al. (2014). How transferable are features in deep neural networks? *NeurIPS*.
4. Loshchilov, I., Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR* (AdamW).
5. Akyon, F. C. et al. (2022). Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. *ICIP* (SAHI).